

Coding Assignments #3 & #4

August 4, 2025

Directions:

1. You will be penalized for not following each of the directions exactly as written below. If you aren't sure about what something means, ask the professor!
2. Put all your work in a single Haskell file.
3. Submit your work (before the deadline) to Canvas.
4. **Important:** You must avoid global helper functions. Keep the scope of your helper functions limited by using `lets` and `wheres`.
5. **Important:** You will be graded on excessive use of helper functions when patterns or guards would make more sense.
6. **Important:** You will lose points **each time** you use the `if` keyword. Use patterns and guards instead!
1. (Call your function **isname**). Using only techniques from chapters 1-3, write a function that tests whether a string is a name or not. Any string is a name if it starts with an uppercase letter. Your function must use a `where` clause that defines a function **issupper** which you use in a guard in your function. Your function use patterns to detect the empty string and produce an error.
2. (Call your function **squarecolor**). Using only techniques from chapters 1-3, write a function that takes as input the file (a-h) and rank (1-8) of a square on a chess board and says whether the square is dark or light. Note that the square a1 is a dark square. Your function must give an error if the square is out-of-bounds. **Hints:** (1) a square is always above, below, to the right of, and to the left of an opposite colored square) (2) your code must test whether the rank modulo 2 is 0 or 1.
3. (Call your function **merge**). Using only techniques from chapters 1-4, write a function that takes two lists of (**each list is already sorted**) and computes a single sorted list containing all the elements of each list, including duplicates.

4. (Call your function **split**). Using only techniques from chapters 1-4, write a function that takes a single list and splits it as evenly as possible into two lists. The signature for the function should be `[a] -> ([a], [a])`, that is, it should produce a pair that contains the two parts of the list.
5. (Call your function **mergeSort**). Using only techniques from chapters 1-4, write a function that takes a single list and sorts that list using the merge sort algorithm and the **split** and **merge** functions you wrote previously for this assignment.